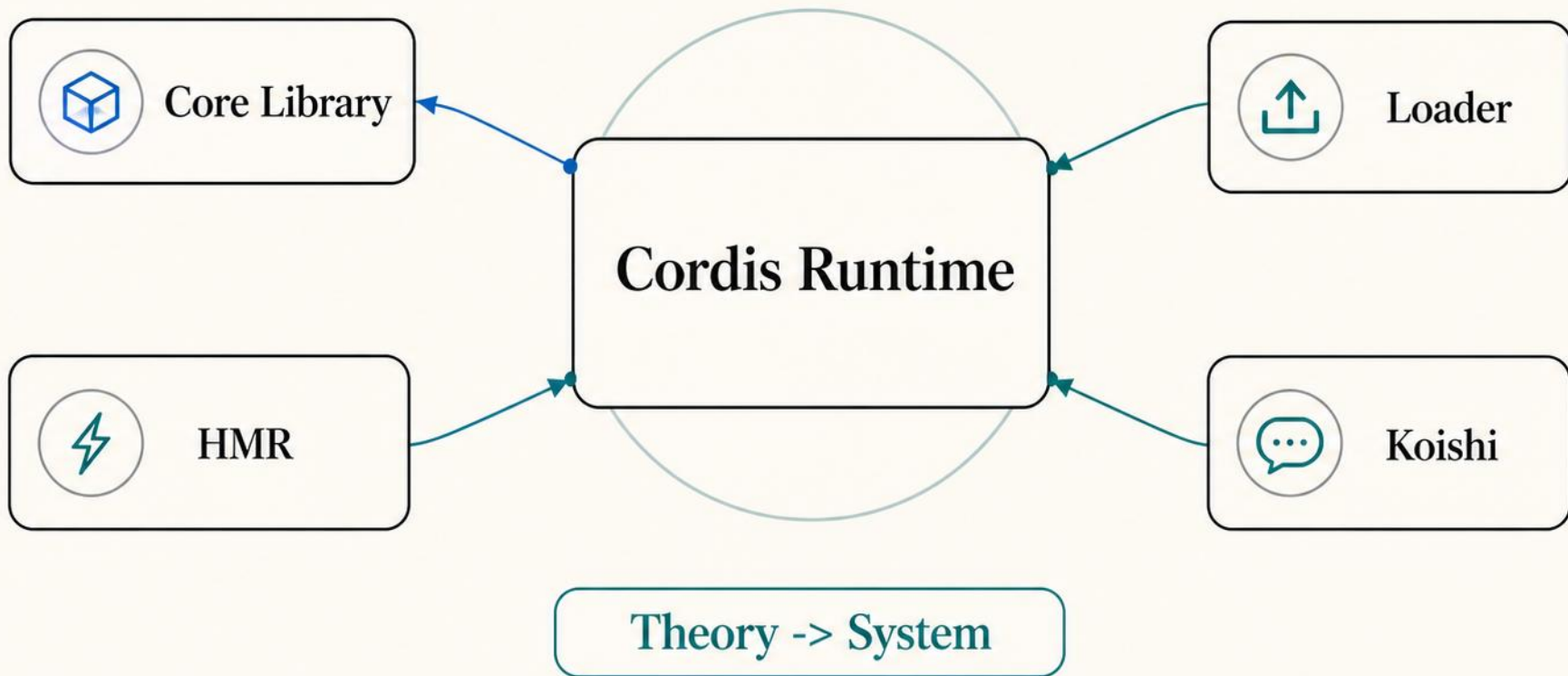


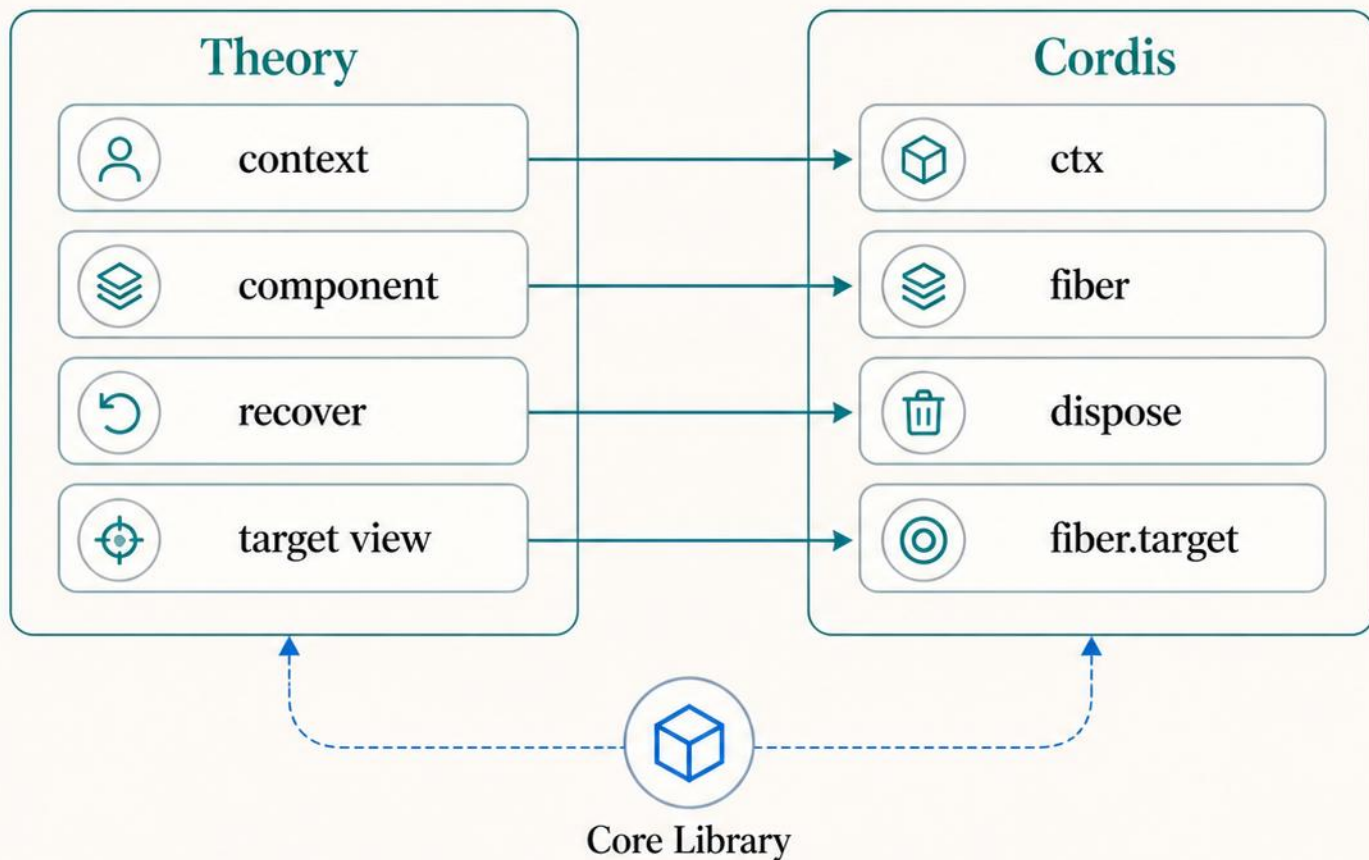
Cordis 实现与系统意义

A Programming Paradigm for Spatiotemporal Composability 解读 · Part 3



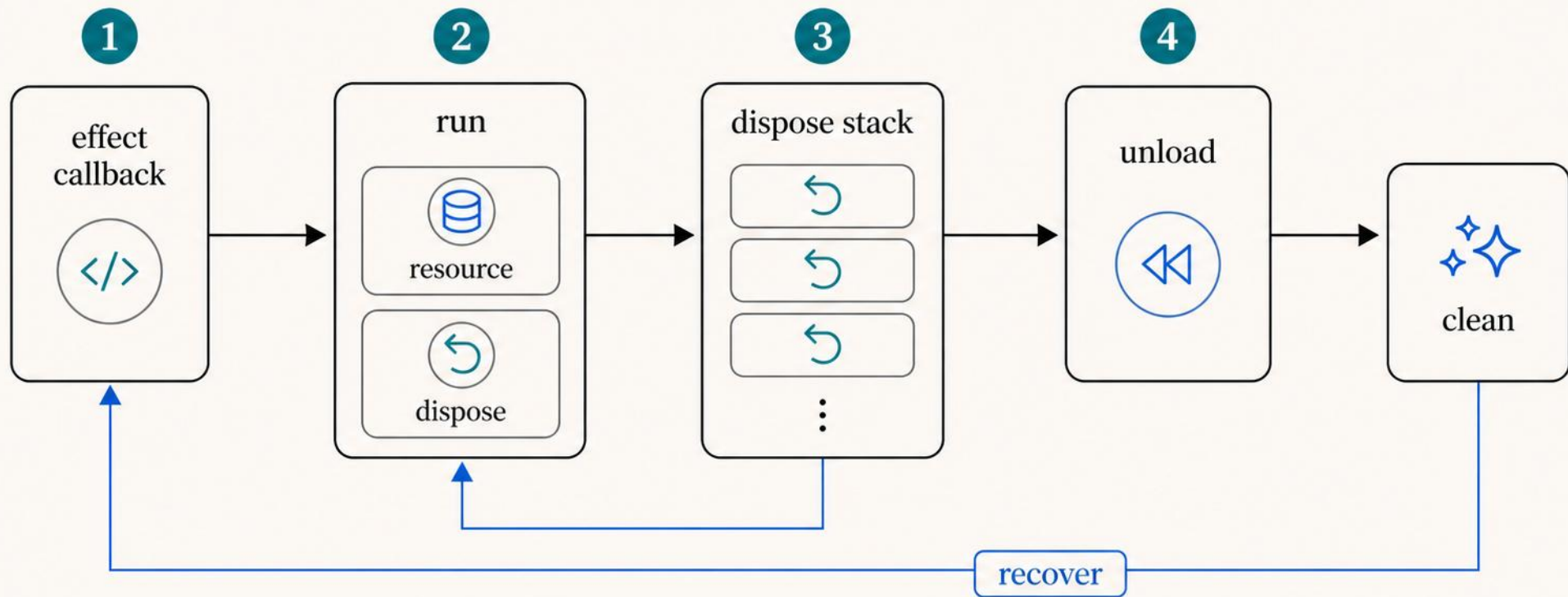
从演算到 Cordis

抽象概念映射成 ctx、fiber、dispose 和 target



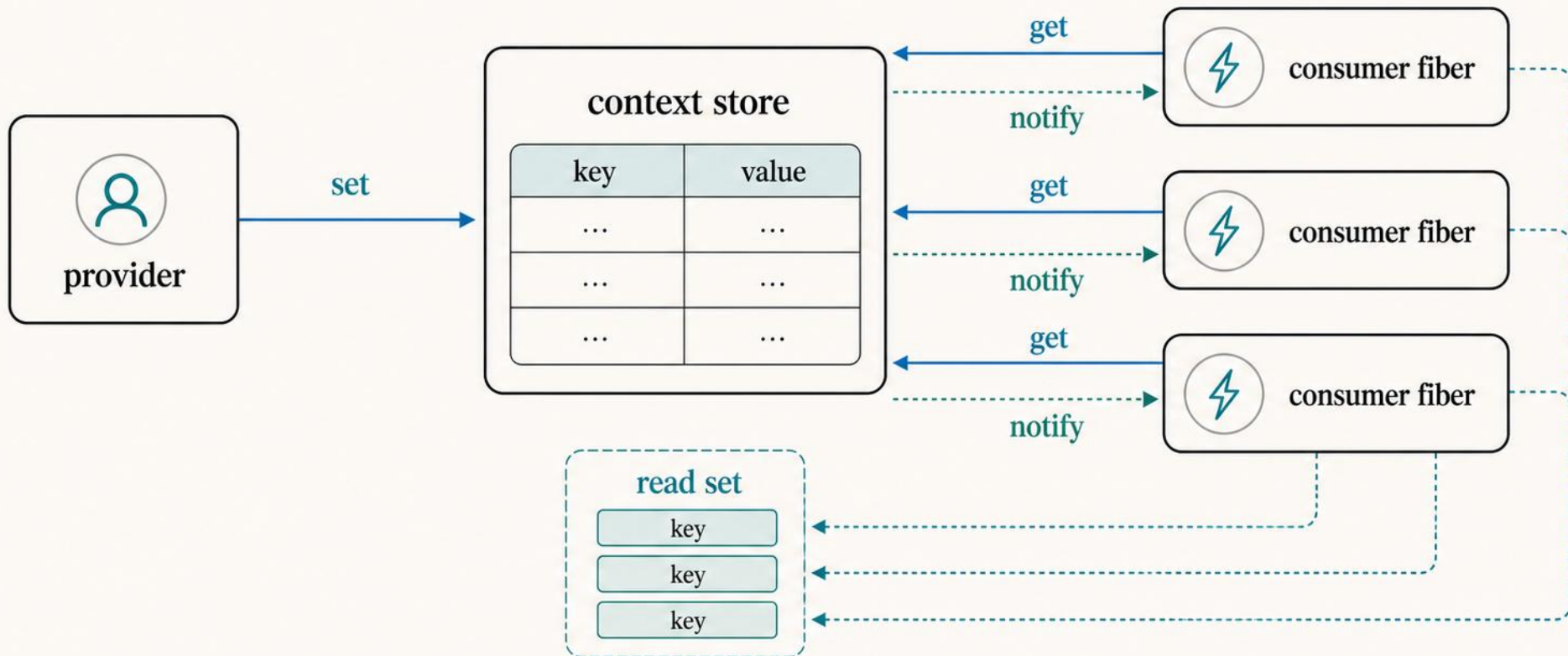
ctx.effect()：让副作用可回滚

effect 返回 dispose, Cordis 把清理函数组合成恢复路径



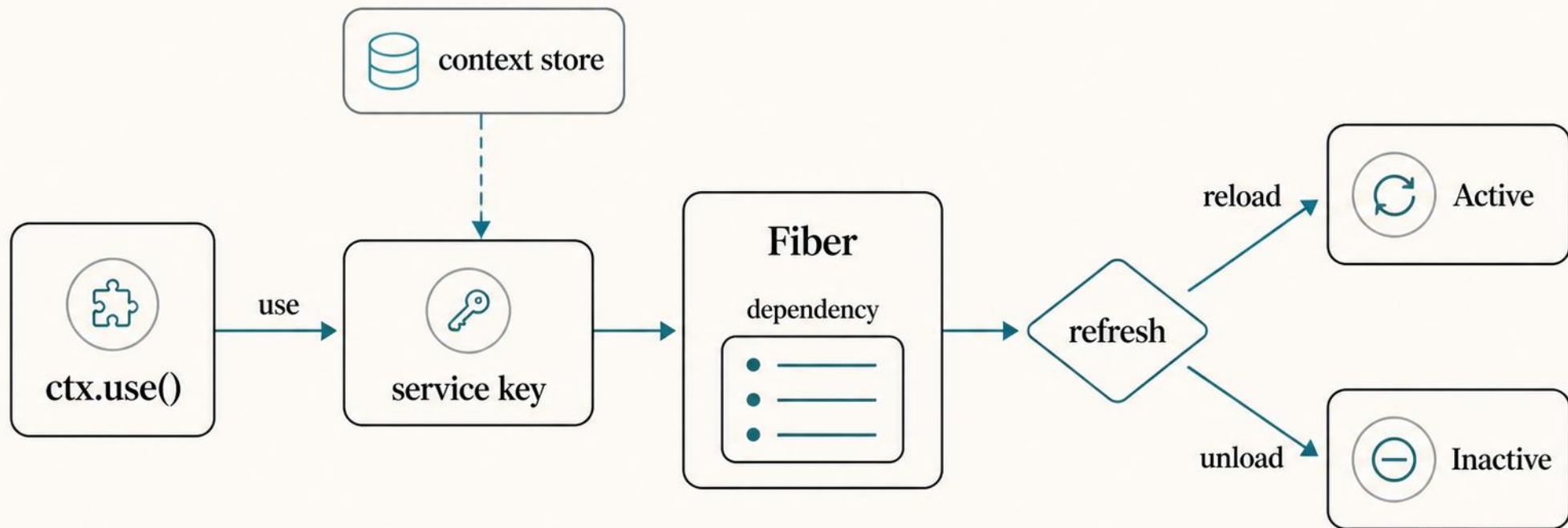
set / get / notify

读上下文会留下 reactive coeffect, 写入时触发依赖刷新



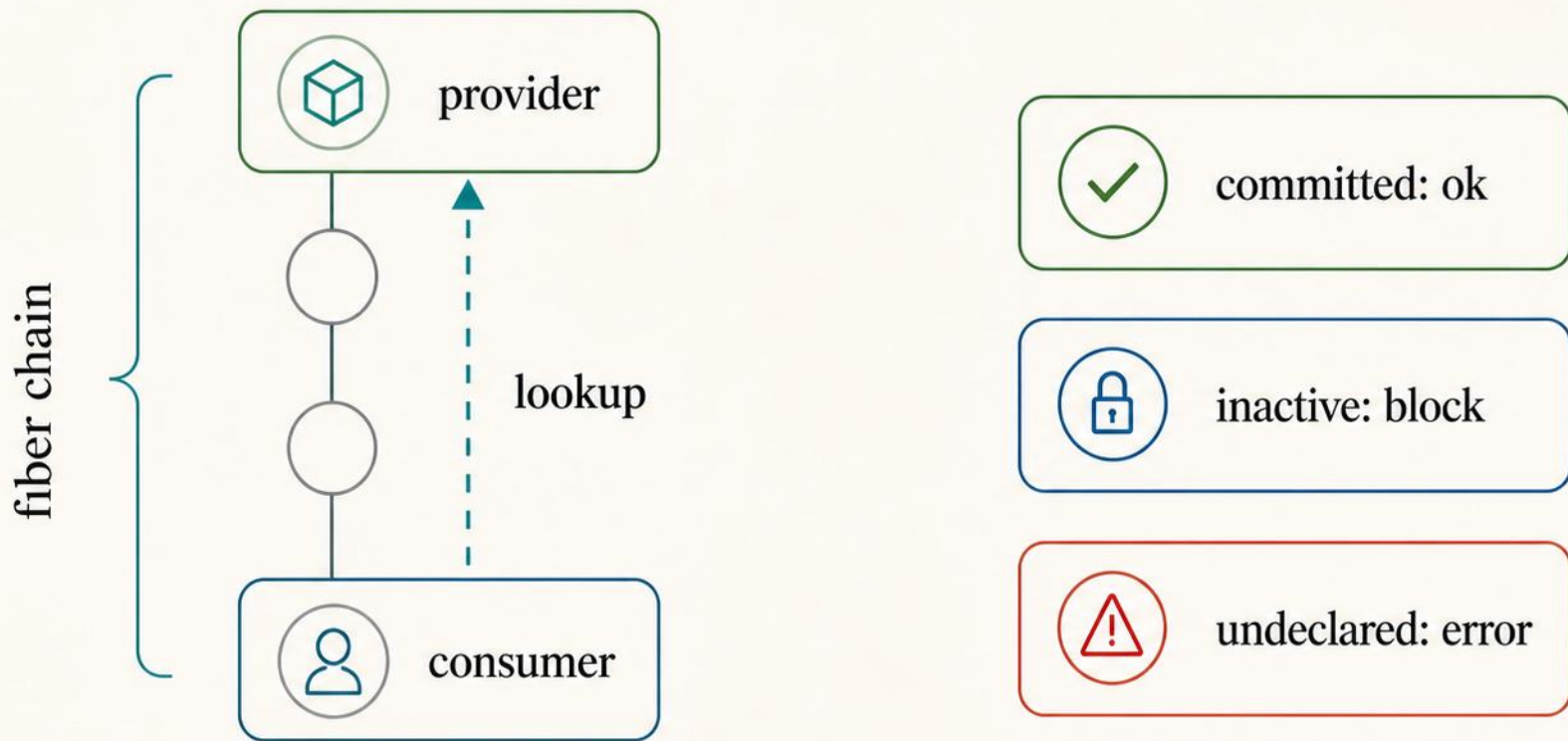
ctx.use(): 服务依赖进入生命周期

use 记录依赖, refresh 决定 reload 或 unload



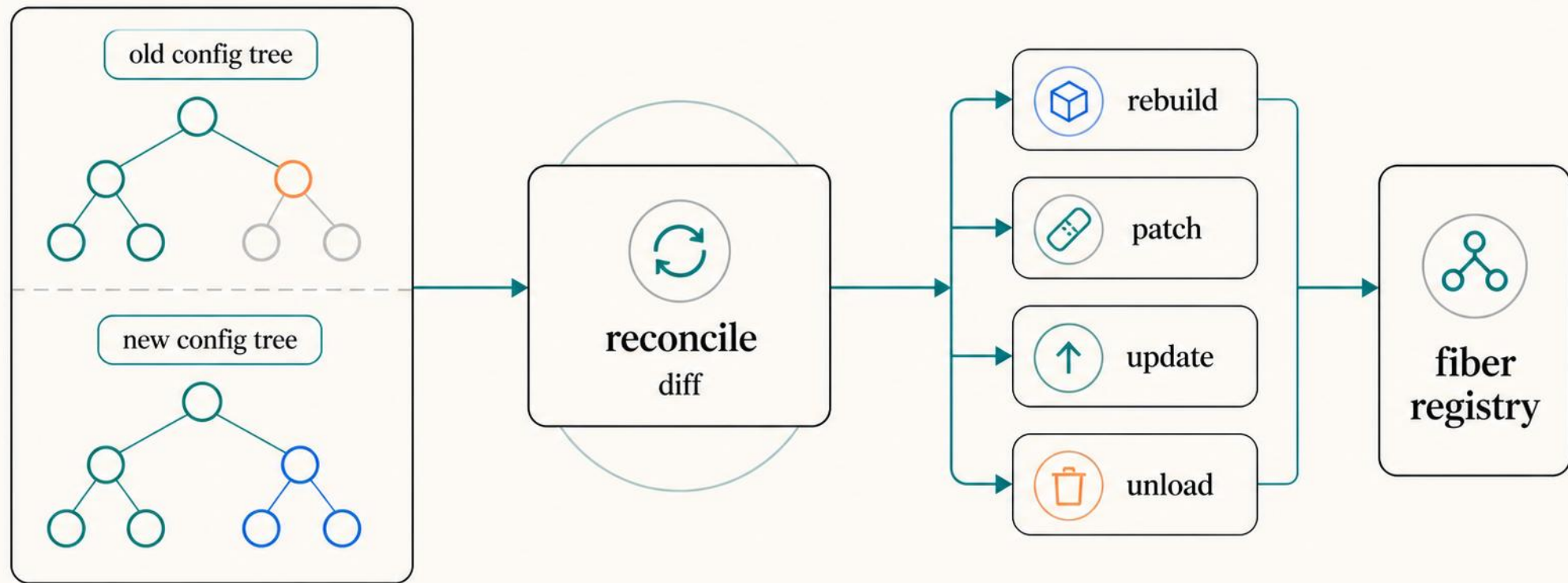
Context Access: 沿 fiber 链查找

只有已提交、仍 active 的 provider 才能被安全访问



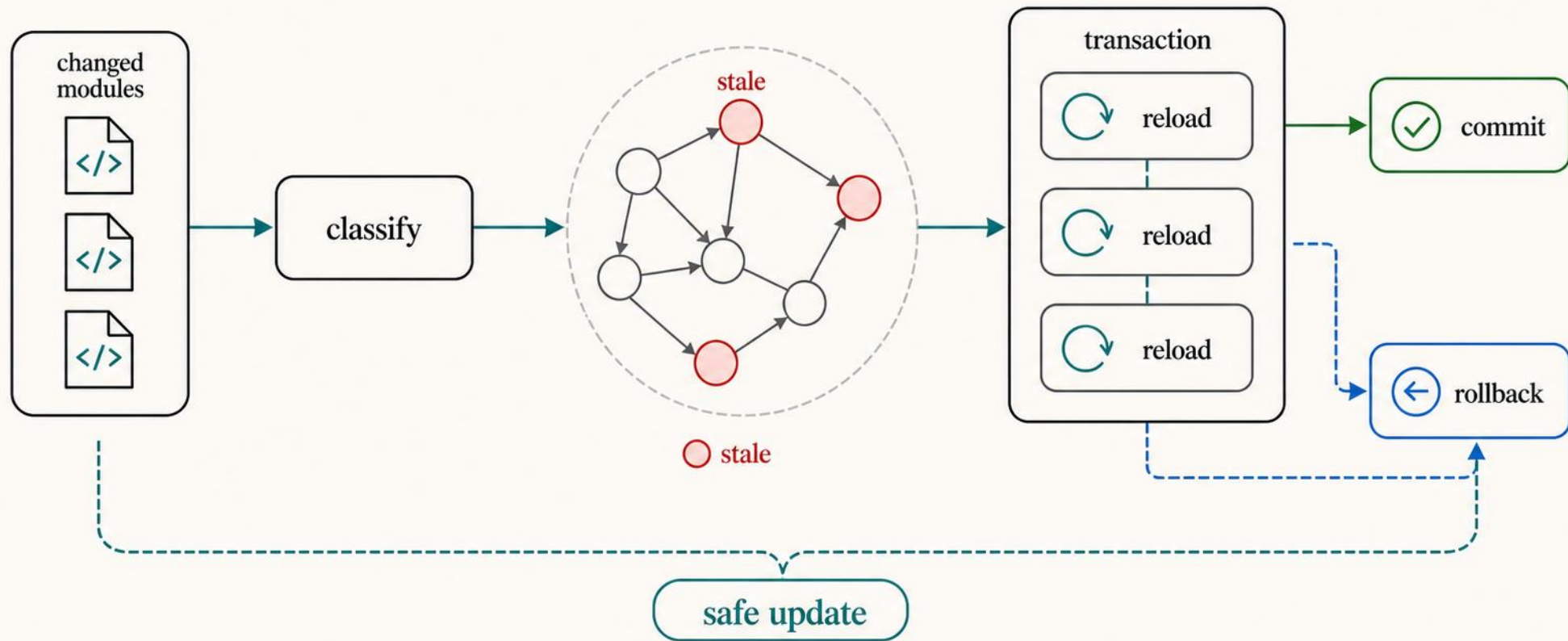
Loader Reconciliation

配置树 diff 后，转成生命周期动作



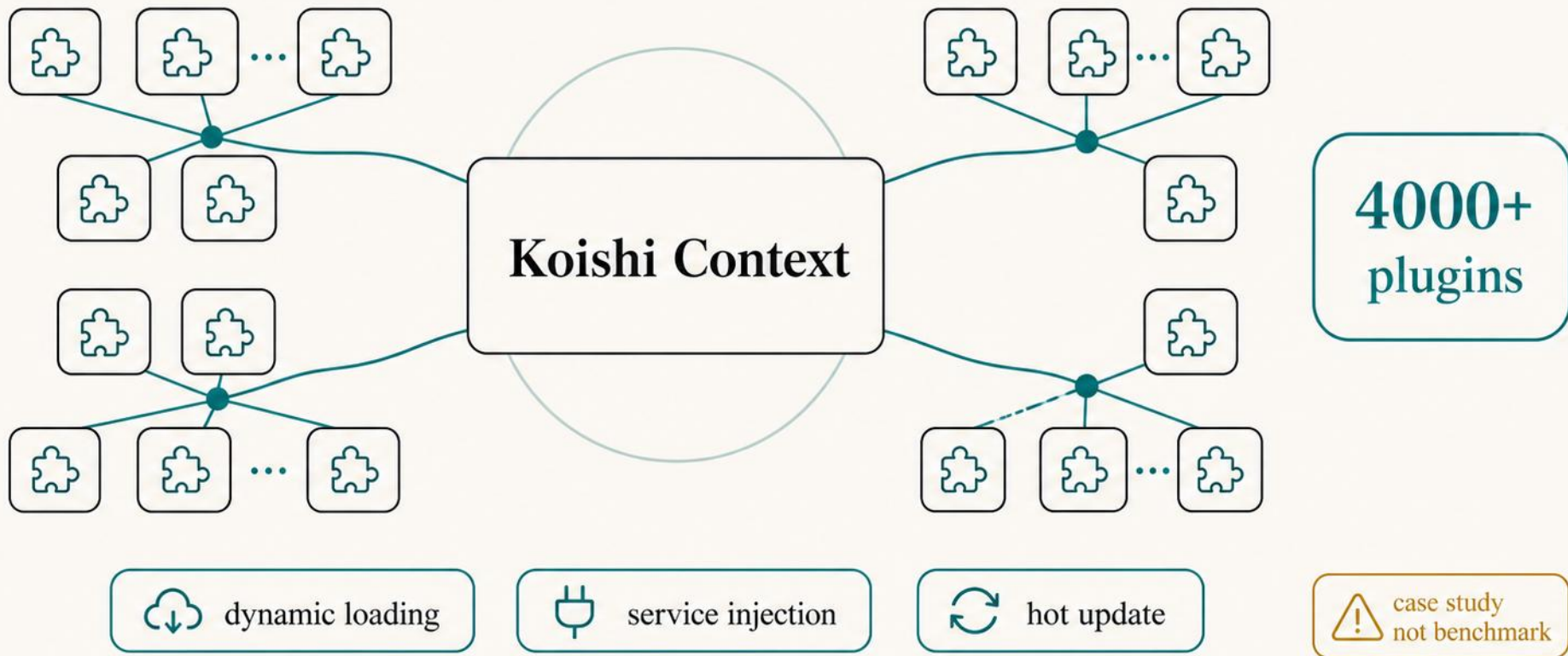
HMR Transaction

热更新被包装成可回滚的生命周期事务



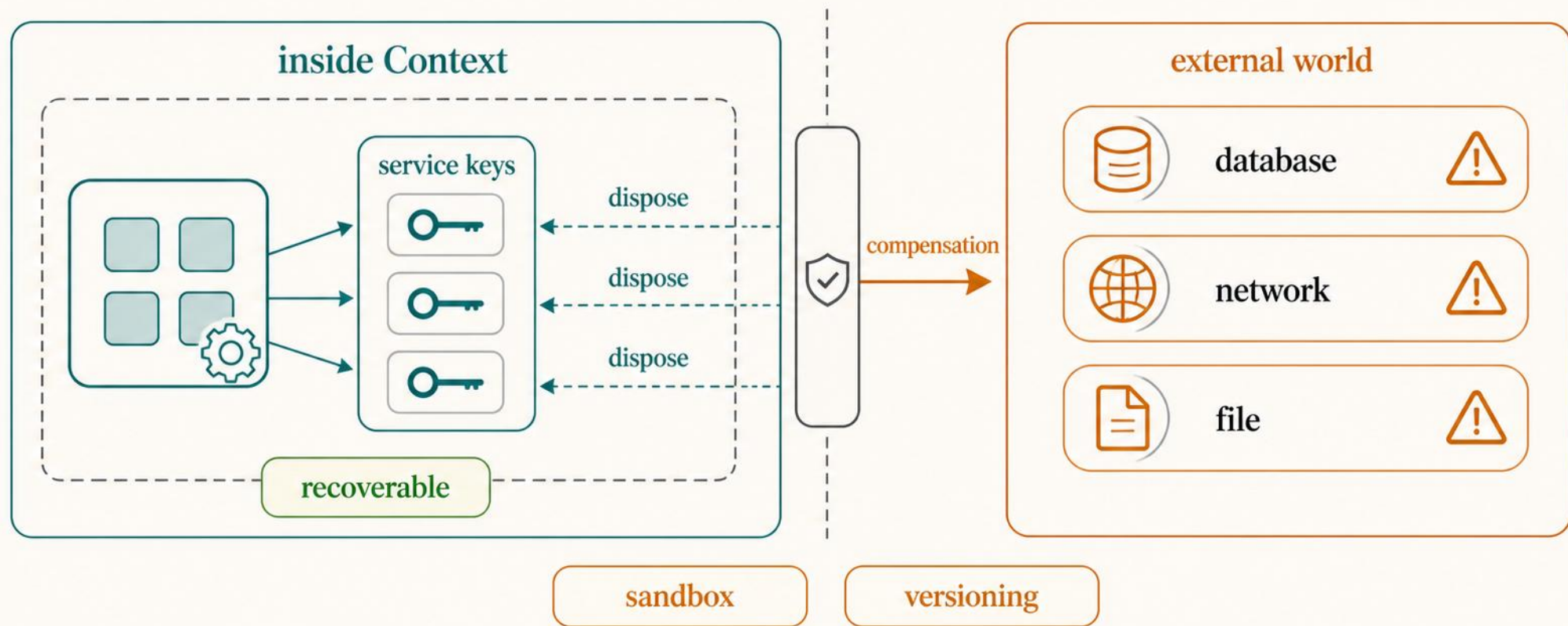
Koishi: 大插件生态里的 Context

4000+ plugins 证明它能支撑长期演化的真实系统



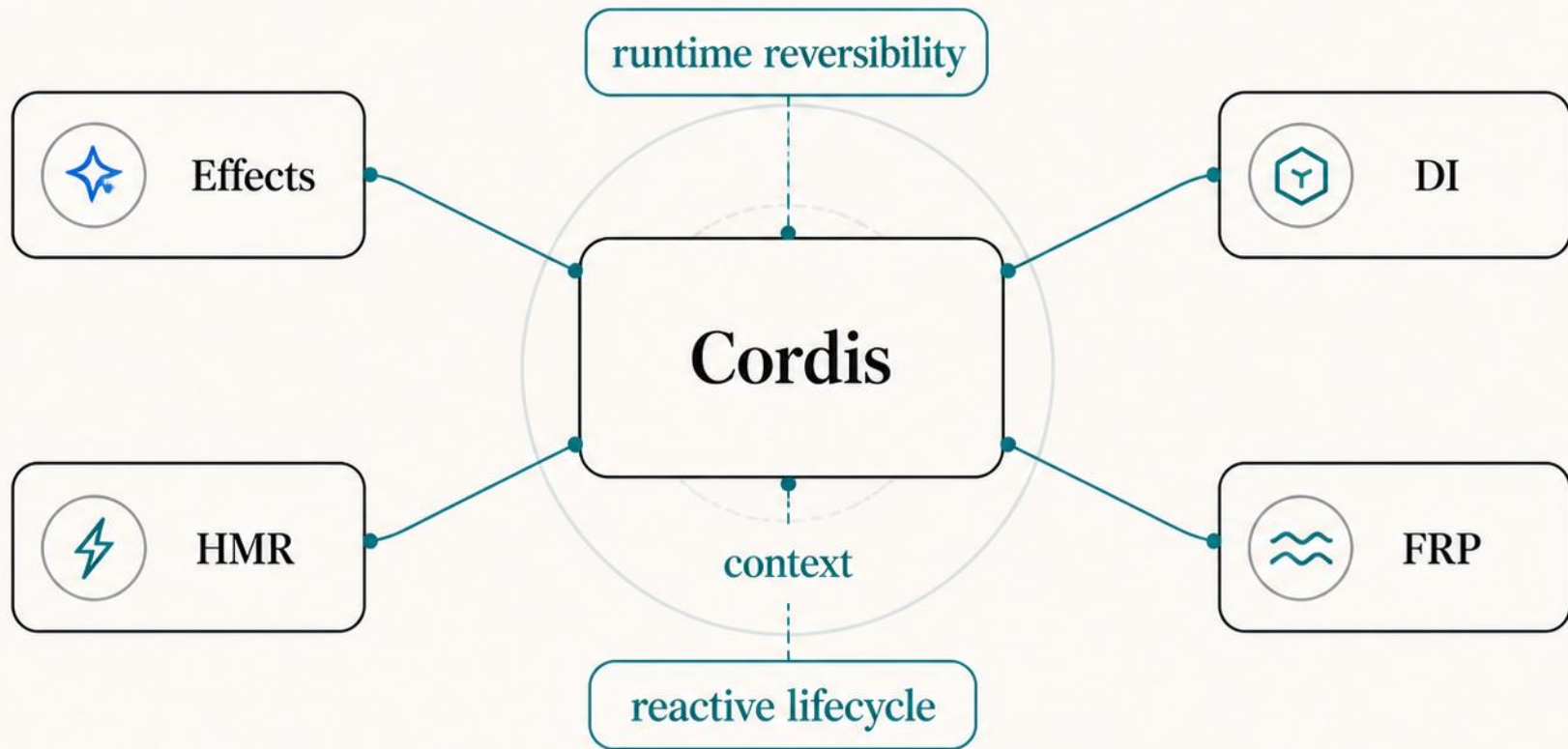
边界：Context 不是万能回滚器

运行时内部可恢复，外部世界需要补偿、沙箱或版本化



它和已有方向的关系

Cordis 不是替代某一类工具，而是把几条线汇到 runtime context



Agent Harness 的想象空间

当组件可恢复、依赖可追踪，agent 生成的系统才可能持续演化

